

Chris Stevens

Professor Neil Kalinowski

CS-499

26 July 2026

Milestone 3 Narrative

The CS-340 Grazioso Salvare Animal Shelter artifact created in June 2025 is used for all three enhancement categories. For this milestone, I focused on the logic used to identify rescue animal candidates. The original dashboard used hardcoded MongoDB filters for Water Rescue, Mountain or Wilderness Rescue, and Disaster or Individual Tracking. This worked for the original project but was limited in that it only returned exact matches. If an animal missed one requirement, it was excluded from the results, even if it matched several other useful criteria.

For this enhancement, I added a rescue candidate scoring algorithm based on weighted criteria instead of only filtering records. The rescue profiles are stored in MongoDB, which makes it much easier to update the categories for weights, age ranges, and breed lists. Each profile includes basic eligibility requirements and scoring rules. For example, the Water Rescue profile requires the animal to be a dog, then scores the animal based on breed, age range, and sex.

As part of this enhancement, I also removed the direct PyMongo integration from the Dash application. The Dash app now requests candidate data directly from the Node/Express API, which applies the rescue profile rules and returns the scored results. This keeps the ranking logic in one place and allows the dashboard to focus on displaying the results.

The following course outcomes are supported by these changes:

- Design and evaluate computing solutions that solve a given problem using algorithmic principles and computer science practices and standards appropriate to its solution, while managing the trade-offs involved in design choices (data structures and algorithms).

The original dashboard used fixed filters, while the enhanced version uses weighted rules to rank animals by how closely they match a rescue profile. One important design choice was separating eligibility requirements from scoring rules. Eligibility requirements help reduce latency by reducing the search set, while the scoring rules rank the animals that remain. This gives each rescue profile more flexibility because not every category needs to use the same rules or weights.

- Employ strategies for building collaborative environments that enable diverse audiences to support organizational decision making in the field of computer science.

The scoring algorithm supports decision-making by giving rescue users a ranked list of candidate animals. Shelter users maintain the animal records, and rescue users use those records to evaluate animals for specific rescue needs. This helps both groups work from the same application while supporting different parts of the process.

- Demonstrate an ability to use well-founded and innovative techniques, skills, and tools in computing practices for the purpose of implementing computer solutions that deliver value and accomplish industry-specific goals (software engineering/design/database).

The algorithm adds value because it makes the application more useful for rescue candidate selection. The scoring profiles are stored as MongoDB documents, which allows the criteria to be updated as rescue needs change. This is more useful than hardcoded filters because the application can be adjusted without rewriting the scoring logic.

The data structures used in this enhancement are MongoDB documents, JavaScript objects, arrays, and sorted lists. Each rescue profile is stored as a document with an array of scoring rules.

The Node/Express API loads the selected profile, builds a query from the eligibility requirements, scores each animal against the rule array, and sorts the results by score. The Dash application fetches those results through the API instead of querying MongoDB directly with PyMongo.

One challenge was deciding which fields should be hard requirements and which fields should contribute to the score. At first, it seemed simple to use all the current filter fields as requirements, but that would have kept the same limitation as the original dashboard. For example, using breed as a hard requirement would exclude an animal that matched the desired age and sex but had a different breed. Keeping breed, age, and sex as scoring rules allows the application to show partial matches and gives the user more candidates to review. Another challenge was thinking through performance. The dataset has around 10,000 records, so scoring every animal on every request could create unnecessary latency. To reduce that issue, the API first applies the eligibility filter, then scores the remaining records, sorts them, and returns a limited number of top candidates. This keeps the algorithm simple while avoiding the need to return every animal record to the dashboard.

This enhancement gave me a chance to apply a pattern I had used in a work project involving computer vision results. That application used a JSON configuration file to evaluate class counts, thresholds, ranges, and bounding box size. I used the same general idea here by storing rescue profile rules in MongoDB and having the API apply those rules to animal records. The final design improves the original artifact by replacing static filters with a profile-driven ranking process. This gives the rescue user a clearer way to compare candidates and makes the application easier to adjust over time.